# PSAI_148

# Avoiding Rust Deadlocks via Visualizing Lifetime

- **Team Name:** Song's research group at Pennsylvania State University
- **Payment Address:** TBD
- **Level:** 3

## Project Overview :page_facing_up:

### Overview

Rust is a new programming language designed to provide both safety guarantees that are like high-level languages and performance guarantees that are like low-level languages. To achieve this design purpose, Rust leverages static compiler checks to rule out severe memory and thread safety issues at the compilation time. At runtime, Rust behaves like C/C++ and could deliver performance that is as good as C/C++. Due to its safety and performance benefits, Rust has seen increasing adoption in building low-level systems software, such as OSs and browsers. Rust is also used to implement many software systems in the Web3 technology stack (e.g., substrate, polkadot, ink!).

Rust's compiler checks are based on a suite of ownership and lifetime rules. The basic rule allows one value to only have one owner variable, and the value is dropped (freed) when its owner variable ends its lifetime. Rust extends the basic rule to allow ownership to be moved and borrowed, while still guaranteeing all accesses to a value to be within its owner variable's lifetime scope. Besides safety checks, lifetime is also used for automated resource management. For example, there is no explicit Unlock() in Rust. A Lock() function call returns a reference to the protected variable, and when the reference ends its lifetime, Rust automatically releases the acquired lock (by implicitly calling Unlock()).

Rust's lifetime rules are complex and different from all other existing languages. It is challenging for Rust programmers to infer where a variable's lifetime ends. As a result, it is not uncommon for programmers to incorrectly identify the location where an implicit unlock is called. When a lock is held longer than programmers' expectation, the same lock may be acquired again or a different lock may be acquired before releasing the acquired lock, leading to a double-lock error or a lock-in-conflicting-orders error.

In our previous work [1], we conducted an empirical study on real-world Rust concurrency bugs. We inspected GitHub commit logs for five Rust applications and five Rust libraries to collect previously fixed concurrency bugs. In total, we found 37 deadlocks due to the misunderstanding of where the implicit Unlock() is called, including 30 double locks and seven locks acquired in conflicting orders. These deadlocks constitute almost all lock-related concurrency bugs (37/38) in our collection. They are all from popular Rust software systems (e.g., Servo, Parity-Ethereum, TiKV, Redox) and have severely hurt the reliability of those systems before they were fixed.

A brief description of the project.

We propose to build an IDE tool for visualizing the lifetime scope of a user-selected Rust variable. We believe our tool can help Rust programmers avoid deadlocks at the development stage. After writing a piece of code involving a mutex, a programmer can select the return value of a locking operation or the locking operation itself (when the return is not saved to a variable). Our tool will visualize the lifetime scope of the return value (i.e., the critical section). The programmer can then inspect whether the end of the critical section is expected. In addition, our tool will conduct deadlock detection for the selected critical section and provide detailed debugging information for identified bugs, such as highlighting blocking operations or function calls leading to blocking operations.

How our tool will be integrated into Substrate/Polkadot?

Both Substrate and Polkadot are implemented in Rust. Previously, double locks or locks in conflicting orders were fixed in Substrate [2, 3]. After applying our prototype, we identified four previously unknown double locks in Substrate or the dependent libraries of Substrate/Polkadot. We reported detected bugs. All of them were confirmed and fixed by developers [4, 5, 6]. We believe our tool will effectively prevent Substrate/Polkadot programmers from making similar mistakes and other types of mistakes our tool will reveal.

Why are we interested in creating this project?

We are interested in building the tool due to three reasons. First, our previous empirical study shows that deadlocks due to the misunderstanding of Rust's lifetime rules are common in Rust programs. Visualizing lifetime can avoid these bugs during the development stage, benefiting the whole Rust community. Second, the misunderstanding of Rust's lifetime rules can also cause memory bugs such as use-after-free and double free. Thus, the proposed tool has the potential to combat memory bugs. Third, the experience of building the proposed tool can inspire similar tools for other programming languages featuring lifetime (e.g., Kotlin).

[1] Boqin Qin, Yilun Chen, Zeming Yu, Linhai Song, and Yiying Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs." In PLDI'2020.

[2] https://github.com/paritytech/substrate/pull/197

[3] https://github.com/paritytech/substrate/pull/6225/commits/61e3b8d53674687790d2b30bc450cd59e09f563d

[4] https://github.com/paritytech/parity-db/pull/8

[5] https://github.com/paritytech/substrate/pull/6277

[6] https://github.com/paritytech/parity-common/pull/396

### Project Details

What have we already done?

We have built a prototype of the proposed tool. Our prototype can visualize a selected variable and conduct double-lock detection. We published a demonstration paper at CCS'2020 to describe the prototype. The paper can be found here: https://songlh.github.io/paper/vr.pdf. We also recorded a video to explain the prototype, and the video can be found here: https://youtu.be/L5F_XCOrJTQ.

We applied the double-lock detection component to Substrate, Polkadot, and ink!. We found four previously unknown deadlocks. One is in Substrate. The other three are in the dependent libraries of Substrate or Polkadot. We reported all the detected bugs. All of them were fixed by developers based on our reporting. The information of the detected bugs is listed as follows:

[PR-1] https://github.com/paritytech/parity-db/pull/8

[PR-2] https://github.com/paritytech/substrate/pull/6277

[PR-3] https://github.com/paritytech/parity-common/pull/396

What are we going to do?

We propose to extend the prototype along four directions:

First, we will integrate our existing implementation of lifetime computation and deadlock detection to rust-analyzer, so that our proposed technique can easily cooperate with different text editors.

Second, we will detect more types of deadlock bugs. Specifically, we will add the detection of locks with conflicting orders and misuse of mutex and non-mutex synchronization primitives (e.g., channel, conditional variable).

Third, we will identify and visualize more blocking operations that can potentially lead to deadlocks in a selected critical section such as receiving from a channel and waiting on a conditional variable.

Fourth, we will implement the visualization functionality by parsing the analysis results generated by rust-analyzer in a text editor (i.e., VS Code) and document our tool. ### Ecosystem Fit

There is no existing project similar to ours.

## Team :busts_in_silhouette:

### Team members

- Name of team leader: Linhai Song
- Names of team members: Linhai Song, Yiying Zhang, Ziyi Zhang

### Contact

- **Contact Name:** Linhai Song
- **Contact Email:** songlh@ist.psu.edu
- **Website:** https://songlh.github.io/

### Legal Structure

- **Registered Address:** Information of our legal structure will be disclosed privately.
- **Registered Legal Entity:** Pennsylvania State University

### Team's experience

The team conducted an empirical study on memory bugs and concurrency bugs in real-world Rust programs. The study was published in PLDI'2020. Through this project, the team has built a comprehensive understanding of common errors made by programmers when coding Rust.

The team built a prototype for the proposed tool. The prototype was published in the demonstration track of CCS'2020, demonstrating the team's capability to build the proposed technique.

The team has another research paper on understanding concurrency bugs in Go published in ASPLOS'2019. The team has more than three years' research experience on concurrency bugs.

Team member Linhai Song has 10 years of expertise in programming analysis, and has published at top programming languages and software engineering conferences (e.g., PLDI, ICSE, FSE, OOPSLA).

Team member Yiying Zhang has conducted various systems research with papers published at OSDI and SOSP.

### Team Code Repos

- LDoctor (ICSE '17)
- Rust-Study (PLDI '20)
- LegoOS (OSDI '18)
- pDPM (ATC '20)
- Pythia (USENIX SEC '20)
- LITE (SOSP '17)
- Hotpot (SoCC '17)

Please also provide the GitHub accounts of all team members. If they contain no activity, references to projects hosted elsewhere or live are also fine.

- https://github.com/songlh

- https://github.com/system-pclub
- https://github.com/WukLab

### Team LinkedIn Profiles (if available)

- https://www.linkedin.com/in/linhai-song-7a3b12120/
- https://www.linkedin.com/in/yiyingzhang/

## Development Status :open_book:

We have built a prototype of the proposed tool. We wrote one paper and recorded one video to describe the prototype.

We applied the bug detection component of the prototype to Substrate, Polkadot, and ink!. We found four previously unknown deadlocks. We reported all the detected bugs and all of them were fixed based on our reporting [PR-1, PR-2, PR-3].

[PR-1] https://github.com/paritytech/parity-db/pull/8

[PR-2] https://github.com/paritytech/substrate/pull/6277

[PR-3] https://github.com/paritytech/parity-common/pull/396

## Development Roadmap :nut_and_bolt:

### Overview

We will integrate the proposed tool into rust-analyzer and demonstrate the virtualization functionality in VSCode, which is an open-source IDE project. We will implement the proposed program analysis by analyzing Rust's MIR.

We divide the project into three milestones. We aim to finish the whole project in three months and achieve a milestone in each month.

- **Total Estimated Duration:** 3 months
- **Full-Time Equivalent (FTE):** 3
- **Total Costs:** financial information will be disclosed privately.

### Milestone 1 — Implement the bug detection component

- **Estimated duration:** 1 month
- **FTE:** 3
- **Costs:** financial information will be disclosed privately.

| Number | Deliverable | Specification |
|---|---|---|
| 0a. | License | BSD |
| 0b. | Documentation | We will provide both inline documentation of the code and a basic tutorial that explains how to run the bug detection component as a standalone tool on terminal. |
| 0c. | Testing Guide | We will include unit tests to ensure the functionality and robustness of our code. We will also include 10 toy programs containing different types of deadlocks to demonstrate the bug detection capability. We will also run this component on the latest version of Substrate, Polkadot, and ink!. We will manually inspect all reported results to count the number of bugs and the number of false positives. |
| 1. | Detecting Conflicting Locks | We will implement a detector that can identify deadlocks due to locks in conflicting orders through analyzing the MIR of Rust programs. |
| 2. | Detecting Misuse of Mutex and Channel | We will implement a detector to identify deadlocks due to errors when using a mutex together with a channel. |
| 3. | Detecting Misuse of Mutex and Conditional Variable | We will implement a detector to identify deadlocks due to mistakes when using a mutex together with a conditional variable. |
| 4. | Docker | We will provide a dockerfile to demonstrate the full functionality of this component. |

### Milestone 2 — Integrate the bug detection functionalities into rust-analyzer

- **Estimated Duration:** 1 month
- **FTE:** 3
- **Costs:** financial information will be disclosed privately.

| Number | Deliverable | Specification |
|---|---|---|
| 0a. | License | BSD |
| 0b. | Documentation | We will provide both inline documentation of the code and a basic tutorial that explains how to install and use the changed rust-analyzer. |

| | | |
|---|---|---|
| 0c. | Testing Guide | We will also use the 10 toy programs designed in the last milestone to test whether the bug detection capability is successfully integrated into rust-analyzer. |
| 1. | Extend Language Server Protocol (LSP) | We will extend LSP to contain the key information related to deadlocks (e.g., the start and the end of a critical section, blocking operations in a critical section). |
| 2. | Change rust-analyzer to emit MIR | We will change the build module of rust-analyzer to emit MIR for our bug detection functionalities. |
| 3. | Conduct bug detection in rust-analyzer | We will change the analysis crate of rust-analyzer to execute the code for bug detection and change the server module to send the detection results out. |
| 4. | Docker | We will provide a dockerfile to demonstrate the full functionality of this component. |

### Milestone 3 — Implement the visualization component

- **Estimated Duration:** 1 month
- **FTE:** 3
- **Costs:** financial information will be disclosed privately.

| Number | Deliverable | Specification |
|---|---|---|
| 0a. | License | BSD |
| 0b. | Documentation | We will provide both inline documentation of the code and a basic tutorial that explains how to install and use the visualization component in VSCode. |
| 0c. | Testing Guide | We will include unit tests to ensure the functionality and robustness of our code. We will also include 10 toy programs to test whether channel operations are correctly identified, whether channel operations are correctly visualized, whether operations on conditional variables are correctly identified, and whether operations on conditional variables are correctly highlighted. |
| 1. | Parse the Extended LSP | We will implement a component to parse the extended LSP and get computed information, such as the scope of a critical section and identified blocking operations. |
| 2. | Highlight Blocking Operations | If a selected variable is the return of a locking operation, besides visualizing the critical section, we will also highlight identified channel operations, conditional variable operations, and locking operations in the selected critical section. |
| 3. | Tutorial Writing | We will write a tutorial and record a video to explain how to use our tool. |
| 4. | Docker | We will provide a dockerfile to demonstrate the full functionality of this component. |

## Future Plans

In the future, we plan to extend the proposed tool along two directions.

First, we plan to extend the proposed tool to cover memory bugs. Our previous empirical study showed that there are also memory bugs due to misunderstanding Rust's lifetime rules, such as use-after-free bugs and double-free bugs. The proposed tool has the potential to help Rust programmers avoid these bugs. Of course, we need to explore what program elements should be visualized for memory bugs.

Second, we plan to conduct a survey to understand what challenges programmers face when understanding Rust's lifetime rules and whether the proposed tool can really help them avoid deadlocks. The survey results will guide us on improving the proposed tool, and broadly speaking, the evolution of Rust.

## Additional Information :heavy_plus_sign:

**How did you hear about the Grants Program?** Web3 Foundation Website

Here you can also add any additional information that you think is relevant to this application but isn't part of it already, such as:

- Work you have already done.

We have built a prototype of the proposed tool. We wrote one paper and recorded one video to describe the prototype.

We applied the bug detection component of the prototype to Substrate, Polkadot, and ink!. We found four previously unknown deadlocks. We reported all the detected bugs and all of them were fixed based on our reporting [PR-1, PR-2, PR-3].

[PR-1] https://github.com/paritytech/parity-db/pull/8

[PR-2] https://github.com/paritytech/substrate/pull/6277

[PR-3] https://github.com/paritytech/parity-common/pull/396

- Wheter there are any other teams who have already contributed (financially) to the project.

No

- Previous grants you may have applied for.

No